

Data Analytics and Machine Learning Methods, Techniques and Tool for Model-Driven Engineering of Smart IoT Services

Armin Moin

Department of Informatics, Technical University of Munich (TUM), Germany

moin@in.tum.de

Abstract—This doctoral dissertation proposes a novel approach to enhance the development of smart services for the Internet of Things (IoT) and smart Cyber-Physical Systems (CPS). The proposed approach offers abstraction and automation to the software engineering processes, as well as the Data Analytics (DA) and Machine Learning (ML) practices. This is realized in an integrated and seamless manner. We implement and validate the proposed approach by extending an open source modeling tool, called ThingML. ThingML is a domain-specific language and modeling tool with code generation for the IoT/CPS domain. Neither ThingML nor any other IoT/CPS modeling tool supports DA/ML at the modeling level. Therefore, as the primary contribution of the doctoral dissertation, we add the necessary syntax and semantics concerning DA/ML methods and techniques to the modeling language of ThingML. Moreover, we support the APIs of several ML libraries and frameworks for the automated generation of the source code of the target software in Python and Java. Our approach enables platform-independent, as well as platform-specific models. Further, we assist in carrying out semi-automated DA/ML tasks by offering Automated ML (AutoML), in the background (in expert mode), and through model-checking constraints and hints at design-time. Finally, we consider three use case scenarios from the domains of network security, smart energy systems and energy exchange markets.

Index Terms—data analytics, machine learning, domain-specific modeling, model-driven software engineering, automl, internet of things

I. INTRODUCTION

Developing software for the *Internet of Things (IoT)* and *smart Cyber-Physical Systems (CPS)* is a relatively complex task. There exist various reasons for this complexity. First, the distributed nature of such systems leads to many challenges. The key concerns and challenges associated with almost any distributed computing system, particularly network-centric service-oriented architectures, include heterogeneity (e.g., with respect to the hardware architectures, operating systems, communication protocols, programming languages and APIs), openness, security and trustworthiness, scalability, failure handling and fault tolerance, concurrency, transparency, quality of service (including availability, reliability, performance and adaptability), as well as privacy [1]. Second, smart CPS, which merge the physical and the virtual worlds using Information and Communication Technologies (ICT) in smart ways [2], pose their own unique challenges, e.g., concerning their *cross-dimension* and cascade effects (i.e., cross-domain, cross-technology, cross-organization and cross-functional),

their *live-dimension* (i.e., re-configuration, re-deployment, live-update, live-enhancement and live-extension), and their *self-dimension*, referred to as *Self-* capabilities* (i.e., self-documenting, self-monitoring/diagnosis, self-optimizing, self-healing and self-adapting/training) [3]. In addition, the Internet is continuously expanding into a pervasive, global and collaborative network of almost every entity/actor (i.e., object or *thing*) surrounding us, including a huge number of sensors and actuators in our smart environments, thus incorporating and integrating them into the IoT. Computing devices networked into the IoT are even much more heterogeneous, e.g., their power supplies/storages, processors, main memories and data storage capacities can have a very broad spectrum. Thus, unprecedented challenges are faced by the designers and developers of such systems [4].

Prior research [3], [5] has proven that *Model-Driven Engineering (MDE)* can be of significant benefit in the domain of IoT/CPS. MDE can hide complexity through abstraction, support communication among stakeholders, and assist with design, early validation/verification/testing, simulation, implementation, and maintenance/evolution in an automated or semi-automated manner. This has motivated *Model-Driven Software Engineering (MDSE)*, particularly *Domain-Specific Modeling (DSM)* with full code generation [6] for the IoT/CPS, as the focus of this PhD study. Moreover, we adopt three use case domains for the creation of our solution, as well as the demonstration of the results and the validation of the proposed approach: network security, smart energy systems and energy exchange markets.

One state-of-the-art domain-specific modeling tool for the IoT/CPS has been delivered by the open source project, *ThingML (Things Modeling Language)* [5], [7]. This doctoral dissertation has targeted the problem of lack of support for Data Analytics (DA) and Machine Learning (ML) at the modeling level of ThingML and other IoT/CPS modeling tools. Given the crucial role of ML in Artificial Intelligence (AI), and the prominent effect of AI on IoT/CPS, this shortcoming in the state of the art represents a critical technological gap that needs to be addressed. Therefore, this doctoral dissertation extends ThingML with DA/ML methods and techniques to facilitate the development of smart IoT services and CPS applications [8], [9]. Concretely, we propose a methodology, a modeling language with extended syntax and semantics, as well as a

number of code generators, based on ThingML.

The rest of this doctoral symposium paper is structured as follows: Section II states our main hypotheses. Moreover, we review the state of the art and mention our expected contributions in Section III. Further, Section IV illustrates our plan for the validation and evaluation methodology. Additionally, a brief description of the results achieved so far is provided in Section V. Finally, we conclude and propose the planned timeline for the completion of the doctoral study in Section VI.

II. HYPOTHESES

We aim to explore *models* in DA/ML and *models* in software engineering, particularly MDSE, in order to find synergies and validate the research hypotheses. This doctoral dissertation has three main hypotheses:

1st Hypothesis: DA/ML concepts can be added to the modeling level of an existing state-of-the-art modeling language and tool for the domain of IoT/CPS, e.g., ThingML, while still supporting the automated generation of the full source code of the target services/applications. Hence, software models in MDSE can be employed to generate ML models and/or deal with them.

2nd Hypothesis: The option of keeping two separate modeling layers, namely Platform-Independent Model (PIM) and Platform-Specific Model (PSM) [10] is feasible and deployable. Specifically, for the platform choices concerning DA/ML, we can support both Java (e.g., via the APIs of WEKA [11] / MOA [12]), and Python (e.g., via the APIs of Scikit-Learn [13] and Tensorflow [14] / Keras [15] / Pytorch [16]) for code generation.

3rd Hypothesis: The workflows and pipelines regarding the DA/ML practices of the users of the resulting tool for the said use case domains can be partially automated. In particular, we can offer Automated ML (AutoML) to assist in the data preparation tasks of time series data, as well as in the choice of ML models/algorithms and hyper-parameter optimization. Also, we can provide hints and live feedback regarding DA/ML practices at design-time via the model-checking constraints, exception handling mechanisms and the model editors.

III. RELATED WORK AND CONTRIBUTIONS

Computer-Aided Software Engineering (CASE): The most relevant work in this regard has been delivered via the open source project *ThingML* [5], [7]. Various research projects, including *HEADS* [17] have contributed to ThingML over the past years. Using the Domain-Specific Language (DSL) of ThingML, one may specify the so-called Heterogeneous and Distributed (HD) services, i.e., IoT services on a higher level of abstraction. Once the model is complete and valid, the code generation framework can produce the full implementation of the target software in a number of programming languages, e.g., Java, Javascript and C, for a range of supported target platforms, e.g., Linux, Arduino and Android, and communication protocols, e.g., HTTP, CoAP and MQTT. The generated code includes the configuration

scripts and is ready to be deployed. As mentioned in Section I, ThingML and other IoT/CPS modeling tools come short of supporting DA/ML practitioners at the modeling level, although DA and ML are quite vital for IoT/CPS.

DA/ML: Increasing the level of abstraction by offering higher level APIs has also been applied to the fields of DA and ML. For instance, there exist a variety of open source libraries and frameworks for ML, such as *Scikit Learn* [13], *TensorFlow* [14], *Theano* [18], *Pytorch* [16], *WEKA* [11], *MOA* [12], *Mahout* [19] and *Mllib* [20]. They offer ML experts a higher level of abstraction. Furthermore, *Keras* [15] and *MXNet* [21] provide yet another higher level of abstraction to practitioners by supporting the APIs of several frameworks and libraries. For instance, Keras supports both TensorFlow and Theano as its backend. Besides, we observed the emergence of a number of workflow designers and tools for partial automation of data analytics pipelines over the past two decades. The *Konstanz Information Miner (KNIME)* [22] is one of the best open-source examples. Note that code generation is also supported to some extent. However, it is not comparable to the systematic approach of the MDSE paradigm. Most importantly, the said workflow designers do not cover any aspect of software beyond DA/ML in their modeling. Hence, they are not sufficient for the IoT/CPS domain.

Model-Based ML: Bishop [23] proposed the idea of *model-based ML*, thus trying to boost ML models to take over the role of software models in MDSE, e.g., by generating the full source code of the target application out of them. That approach has been implemented in the open source framework *Infer.NET* [23], [24], which supported the probabilistic programming paradigm. The user of Infer.NET should specify a Probabilistic Graphical Model (PGM) via the provided DSL. The framework could then generate the source code of the target software application out of the PGM. However, we argue that this approach is highly restrictive, since PGMs are not expressive enough to let the user specify smart IoT services / CPS applications sufficiently. Hence, our proposed approach is vice versa, i.e., we enhance software models to become capable of producing and dealing with ML models. Further, Infer.NET is inflexible, both in terms of the choice of the ML models and algorithms (limited to PGMs, while other ML models, such as neural networks are currently widely used in industry), and with respect to the code generation (i.e., the generated code can only be in C#). Last but not least, Breuker [25] conducted an exploratory analysis of DSLs for ML and proposed an initial conceptualization of a DSL, based on Infer.NET.

Expected Contributions: We propose a novel approach, called *ThingML+*, which addresses the above-mentioned shortcomings of the related work (see Table I). In particular, we adopt the idea of model-based ML [23], but, as stated above, in a very different way. Instead of using ML models as software models, we enhance software models to become capable of supporting ML, e.g., by generating the appropriate ML models for the task at hand, and training them automatically using existing and/or incoming data. In addition, the user may bring

his or her own trained ML model, e.g., an Artificial Neural Network (ANN), which is already trained on a dataset, and simply connect that to the software model. This is called a mixed MDSE/non-MDSE approach, since the software model may not include all the necessary details to be able to generate the ANN and the source code that is necessary to train it on existing or new data instances in the future. Finally, we implement our proposed approach in an open source tool, called *ML-Quadrat* [26]. Similar to ThingML, ML-Quadrat is also built on top of the Eclipse Modeling Framework (EMF). From the research perspective, we validate the three hypotheses mentioned in Section II and evaluate the proposed approach.

TABLE I
COMPARISON OF THE STATE OF THE ART WITH THE PROPOSED APPROACH
(THINGML+)

	CASE / MDSE w. full-code gen.	DA / ML w/o AutoML	IoT / CPS
ThingML	✓		✓
KNIME, WEKA, TensorFlow, etc.		✓	
INFER.NET	✓	✓	
ThingML+	✓	✓	✓

IV. VALIDATION AND EVALUATION PLAN

Newman [27] highlighted four broad categories for engineering research: (i) Enhanced Analytical Modeling Techniques (abbreviated as EM), (ii) Enhanced Solutions (abbreviated as ES), (iii) Radical Solutions (abbreviated as RS), and (iv) Enhanced Tools and Methods (abbreviated as ET). Although his work originated from the field of Human Computer Interaction (HCI), it was not specific to HCI, but rather for any engineering research work. The three hypotheses mentioned in Section II indicate that our research is primarily about the latter class, i.e., enhancement of tools and methods. Hence, as suggested by Newman [27], we shall validate the proposed approach by implementing our modeling tool and methods, thus showing the feasibility, e.g., for the selected use case domains, through a number of working examples. Note that we employ the use case domains of network security and smart energy systems for creating our solution (see Section V). However, for the validation and evaluation, we use the use case domains of smart energy systems (with different datasets) and energy stock markets.

Furthermore, we shall validate the usability of the proposed approach. This will be done in two ways. First, we shall demonstrate that the automatically generated code performs on average better or at least as good as the state of the art (i.e., manually developed software). In fact, we expect our approach to outperform the state of the art, since the proposed AutoML functionalities shall lead to performance leaps, e.g., due to the optimized data preparation efforts, as well as model/algorithm and hyper-parameter choices for ML. Second, we shall also study the user experience and confirm that the intended audience, who are mostly software

developers without deep knowledge and skills in DA/ML or specific target IoT platforms, are more satisfied with using our proposed solution than traditional approaches, i.e., manual software development. For the validation of the usability and user experience, we shall conduct an empirical study with human participants.

Therefore, we shall validate each of the said hypotheses as follows: (i) For the first hypothesis, we shall show that the source code that will be generated out of each model instance does not require any further manual changes, and is ready to be deployed and executed on the respective target platforms. (ii) Concerning the second hypothesis, we shall illustrate that model instances may be Platform-Independent Models (PIM) or Platform-Specific Models (PSM) via examples. Usually, we let the user of our tool import a PIM into a PSM and augment it with the platform-specific details to make it a complete model instance for code generation. (iii) Regarding the third hypothesis, we shall compare the performance and efficiency of the automatically generated code in the AutoML mode with the manually developed code. In particular, we shall compare the accuracy, precision, recall and F1-measure in both cases and confirm that the AutoML functionalities improve those metrics.

Finally, GQM (Goal, Question, Metric) [28] provided a measurement model on the three levels of conceptual (Goal), operational (Question) and quantitative (Metric). We shall consider this approach in order to devise our concrete validation plan.

V. PRELIMINARY RESULTS

So far, we have published two peer-reviewed papers for this dissertation. The first one, *ThingML+* (2018) [8] was a position paper (3 pages), which illustrated the core concept. Moreover, the second one, *ThingML2* (2020) [9] was a poster together with a companion extended abstract (2 pages), which demonstrated the proof of concept and the validation of the first hypothesis. In the following, we present the current status of our ongoing research and development. Our research prototype is available as open source software with Apache License 2.0. Please check out our repository on Github [26].

Use Cases

We focus on three use case domains as follows. The main use case domain is smart energy systems, which will be used for both creating our solution and validating it, but with different datasets. However, the network security use case shall be additionally employed for creating the solution, whereas the energy exchange markets use case shall further validate the proposed approach.

Network Security: Distributed Denial-of-Service (DDoS) attacks are among the most prevalent types of malicious cyber activities to prevent legitimate access to Internet-based services. We consider a simple client-server interaction, where the server simply waits for ping messages from clients and responds to each ping message with a pong message. We enhance this ping-pong example using ML. Concretely, we

empower the server with the ability to check whether the client is likely to be an attacker. If this is the case, the server will simply ignore the ping message of the client. Otherwise, the server may send out the pong message in response to the client. The ML model, which is trained using an existing dataset of IP addresses and timestamps of previously received ping messages over a time period (e.g., several months), uses the IP address and the timestamp of the ping messages to make predictions. It predicts a boolean value for each ping message. If the sender (client) is more likely to be an attacker, the predicted boolean value will be true. Otherwise, it will be false. The pre-existing dataset must be sufficiently long to expect meaningful predictions by the ML model. This use case serves for illustration purpose and is not a real-world use case scenario (unlike the other two use cases).

Smart Energy Systems: We are interested in the problem of *Non-Intrusive (Appliance) Load Monitoring (NIALM/NILM)*, also known as *energy disaggregation*, in the context of *smart grids*. Smart grids hold as an example for smart CPS, where the energy producers, energy consumers and the so-called *prosumers* (i.e., those participants, who may play both roles) can be connected via the IoT, in order to optimize the overall demand and supply in the network, and make the entire grid more robust and reliable. In many cases, it is necessary or at least advantageous for the system to find out which appliance is active at each household or building that is connected to the smart grid at each point in time, and how much energy it consumes. For instance, if the system is supposed to consider certain discounts and incentives for those consumers, who refrain from using their *energy-hungry* appliances, e.g., air conditioners or cryptocurrency mining servers during the *peak demand* times of the grid, and have agreed to use certain appliances, e.g., dish washers, only in the *off-peak* times, e.g., mid-nights, it must be possible for the system to verify whether they have indeed respected the agreements or whether any possible violations have occurred. Measuring the exact power consumption of an individual appliance is only possible, if one attaches a sensor/counter, known as a *smart meter* to it. That is called the *intrusive* approach, since new connected devices from the outside must be added to the household. However, it is not only costly for the owner, but can also lead to inconveniences, as well as security and privacy concerns of the occupants. In contrast, the *non-intrusive* approach employs algorithms and methods to disaggregate the total power consumption signal into individual loads, thus *virtually* sensing/counting the estimated power consumption of each appliance at each time. Some energy disaggregation approaches may also predict the types of appliances, which reside in each household, by comparing their power consumption patterns with certain *signatures* of common home/office appliances. We use three public datasets for NIALM. The first one is the REDD dataset [29] from the USA, whereas the second and the third ones are the UK-DALE [30] and the REFIT [31] datasets from the UK. Fig. 1 depicts the architecture of our example for NIALM. This example involves three *things*: the energy disaggregation server of the

smart grid, the DA/ML server of the smart grid, and the smart home of the energy consumer. The energy disaggregation server delegates the actual task of performing NIALM to the DA/ML server. Further, we illustrate the state machine model of the behavior of the energy disaggregation server in Fig. 2. The state machines concerning the behavioral models of the other two *things* are not shown here. Finally, the datasets that are used for creating the solution and validating the proposed approach are mutually exclusive.

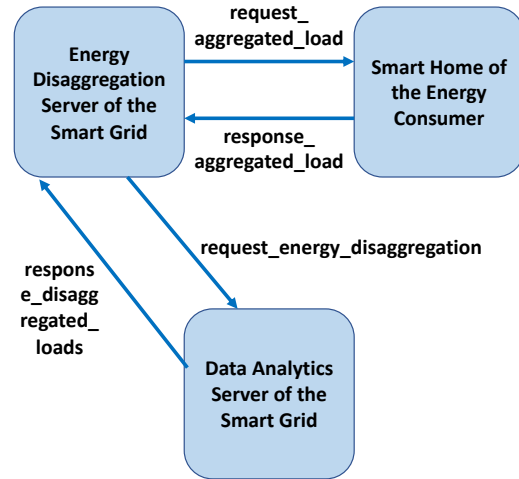


Fig. 1. The architecture of our example for NIALM.

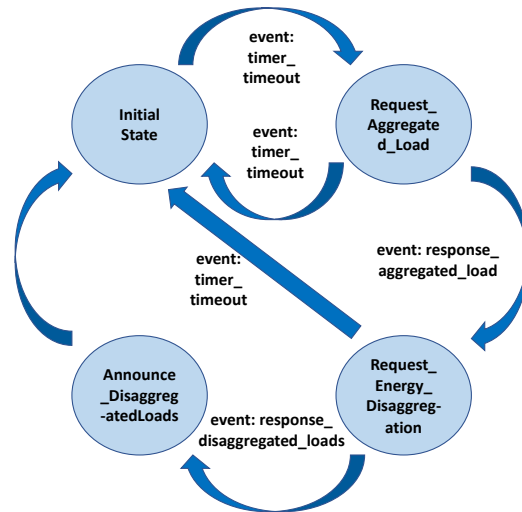


Fig. 2. The behavioral model of the energy disaggregation server.

Energy Exchange Markets: We use the open data from energy exchange markets, e.g., the European Energy Exchange (EEX) to create a prediction service for the electricity prices in the near future. This use case domain shall only be used for the validation of the proposed approach. The ML model employed here will be a sequence-to-sequence model, trained using the historic data of the dynamic electricity prices over

a period of time. Given a list of current electricity prices in the market, the prediction service shall forecast the electricity prices in the near future, e.g., for the next day.

Our Research Methodology

First, we adopt the modeling language and tool provided by the ThingML project. Second, we manually develop a number of examples for the target IoT services. To this aim, we concentrate on the network security and the smart energy systems use cases. Based on that, we extract the required concepts that need to be added to the existing modeling language and tool of ThingML. Furthermore, we explore relevant standards, such as the Predictive Model Markup Language (PMML), as well as the open source libraries and frameworks for ML, specifically Scikit-Learn, Keras, TensorFlow and WEKA to select the additional concepts and platform-specific APIs that shall be added to the meta-model and the tool. Moreover, we intend to employ additional principles and guidelines in the modeling community, e.g., [32] and [33], to ensure that we have a proper and complete DSL.

Software Development Methodology at Design-Time

A distributed system in ThingML is modeled as a set of components (i.e., *things*), which communicate with each other using asynchronous message passing. In order to design a new IoT service / CPS application using ThingML, one shall specify the following: (i) The ports for message passing, the messages together with their parameters, and the properties (i.e., variables) for each *thing*. (ii) The UML-like state machine (state diagram or statechart), which models the behavior of each *thing*. ThingML offers an imperative platform-independent action language to support the event-driven programming paradigm on the state machines. (iii) The configuration for the entire service/application, which comprises the instances of the defined *things* and the desired connections among their ports. We extend the methodology of ThingML to support DA/ML. Concretely, we let the user of our tool dedicate certain *things* or *thing fragments* (i.e., sub-components of *things*) to DA/ML tasks. Therefore, for such *things*, the user shall also specify an additional block concerning DA/ML, before the state machine that specifies the behavior of the *thing*. Moreover, *things* dedicated to DA/ML typically have the following states: (i) data preprocessing, (ii) training, (iii) ready for prediction, (iv) predict. If the user decides to bring his or her own trained ML model, states (i) and (ii) may be skipped.

Domain-Specific Modeling Language

We extend the DSL of ThingML with the following items: (a) An optional DA/ML block for each *thing*, which can, e.g., include the list of properties (variables), which shall be considered as the ML features/attributes, and the choice of ML model and algorithm in the expert mode. If the user is not an expert in ML, the mode shall be set to AutoML instead of the expert mode. (b) New options in the imperative action language that may be employed for event-driven programming.

The new options include *DA save action*, *DA preprocess action*, *DA train action* and *DA predict action*. They can be used in the conditions for the transitions of the state machines of *things* responsible for DA/ML. Fig. 3 demonstrates part of the abstract syntax of our DSL [9].

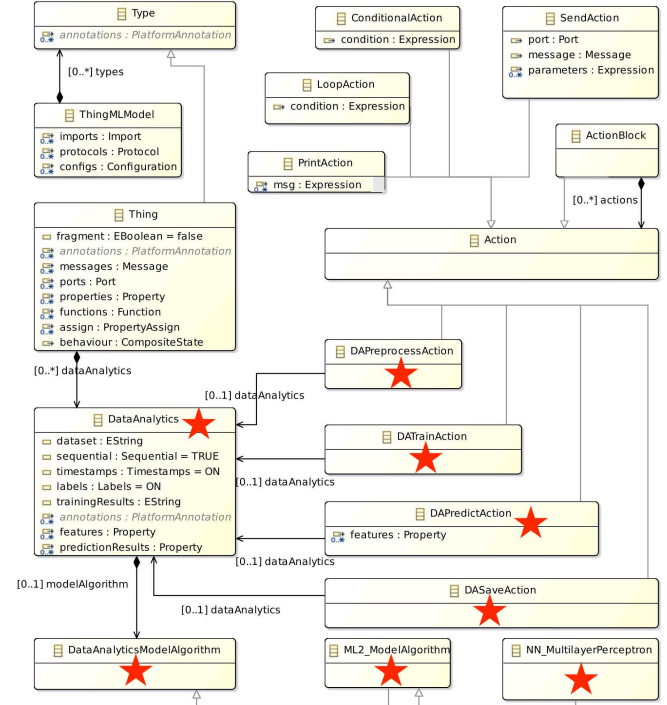


Fig. 3. Part of the abstract syntax of the DSL. The red stars mark some of our extensions of the ThingML meta-model [9].

Code Generators

Currently, we support code generation in Python for ML, using the APIs of Scikit-Learn and Keras (with the Tensorflow backend). The generated Python code is seamlessly integrated with the Java code using the *Process Builder* API of Java. Moreover, the generated code can be easily installed and deployed using Apache Maven. All the required configuration scripts are generated out of the software model instance in an automated manner.

Model Editors

We support both the customized Xtext-based textual model editor of ThingML, as well as an EMF-based graphical tree model editor. Furthermore, a web-based model editor is in development. A preliminary textual version of the web-based editor is already available.

VI. COMPLETION PLAN

Until the Doctoral Symposium, we will finish the implementation and validation. After the event, we will have at least 9 months for applying the feedback that is expected to be received at the event, writing the dissertation and preparing

for any possible resubmission of the planned research papers. Currently, the following peer-reviewed research papers (6+ pages) are planned for the completion of the doctoral study: (i) Validation of the first hypothesis (without the empirical study); (ii) Validation of the meta-model, the second hypothesis and support for a standard Model Interchange Format (MIF), such as PMML, PFA (Portable Format for Analytics) or ONNX (Open Neural Network Exchange) for the platform-independent layer; (iii) Validation of the third hypothesis (without the empirical study); (iv) The empirical study using human participants concerning the user experience; (v) Survey and benchmarking of the state of the art for the use case domain of smart energy systems, specifically NIALM (optional journal article).

ACKNOWLEDGMENTS

The advisor, second advisor and mentor of this work are Stephan Günemann, Atta Badii and Marouane Sayih, respectively. The author would like to sincerely thank them for their kind supports. This work is partially funded by the German Federal Ministry of Education and Research (BMBF) through the Software Campus program (project ML-Quadrat). The author would like to thank Stephan Rössler and his colleagues at Software AG for their help.

REFERENCES

- [1] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. USA: Addison-Wesley Publishing Company, 2011.
- [2] E. Geisberger and M. Broy, Eds., *Living in a networked world. Integrated research agenda Cyber-Physical Systems (agendaCPS)*, ser. acatech STUDY. Munich, Germany: Herbert Utz Verlag, 2014.
- [3] B. Schaetz, "The role of models in engineering of cyber-physical systems – challenges and possibilities," in *CPS20: CPS 20 years from now - visions and challenges*, ser. CPS Week, 2014.
- [4] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787 – 2805, 2010.
- [5] N. Harrand, F. Fleurey, B. Morin, and K. E. Husa, "ThingML: A language and code generation framework for heterogeneous targets," in *Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS '16, 2016.
- [6] S. Kelly and J.-P. Tolvanen, *Domain-Specific Modeling: Enabling Full Code Generation*, 1st ed. Wiley-IEEE Computer Society Pr, 2008.
- [7] "ThingML," <https://github.com/TelluIoT/ThingML>, accessed: 2020-04-29.
- [8] A. Moin, S. Rössler, and S. Günemann, "ThingML+: Augmenting model-driven software engineering for the internet of things with machine learning," in *Proceedings of the 2nd International Workshop on Model-Driven Engineering for the Internet of Things (MDE4IoT)*, 2018.
- [9] A. Moin, S. Rössler, M. Sayih, and S. Günemann, "From things' modeling language (ThingML) to things' machine learning (ThingML2)," in *Proceedings of MODELS 2020 Satellite Events (Poster Companion / Extended Abstract)*, the ACM / IEEE 23rd International Conference on Model Driven Engineering Languages and Systems (MODELS), 2020.
- [10] "Model Driven Architecture (MDA), MDA Guide rev. 2.0," Object Management Group, Boston, MA, USA, Standard OMG Document ormsc/14-06-01, 2014. [Online]. Available: <https://www.omg.org/cgi-bin/doc/ormsc/14-06-01>
- [11] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, "The weka data mining software: An update," *SIGKDD Explor. Newsl.*, vol. 11, no. 1, p. 10–18, Nov. 2009.
- [12] A. Bifet, G. Holmes, R. Kirkby, and B. Pfahringer, "Moa: Massive online analysis," *J. Mach. Learn. Res.*, vol. 11, p. 1601–1604, Aug. 2010.
- [13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [14] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: <http://tensorflow.org/>
- [15] F. Chollet *et al.*, "Keras," <https://keras.io>, 2015.
- [16] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," in *NIPS-W*, 2017.
- [17] "HEADS," <https://cordis.europa.eu/project/id/611337>, accessed: 2020-02-08.
- [18] Theano Development Team, "Theano: A Python framework for fast computation of mathematical expressions," *arXiv e-prints*, vol. abs/1605.02688, May 2016. [Online]. Available: <http://arxiv.org/abs/1605.02688>
- [19] "Apache Mahout," <https://mahout.apache.org>, accessed: 2020-09-12.
- [20] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "Mllib: Machine learning in apache spark," *J. Mach. Learn. Res.*, vol. 17, no. 1, p. 1235–1241, Jan. 2016.
- [21] T. Chen, M. Li, Y. Li, M. Lin, N. Wang, M. Wang, T. Xiao, B. Xu, C. Zhang, and Z. Zhang, "Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems," 2015.
- [22] M. R. Berthold, N. Cebon, F. Dill, T. R. Gabriel, T. Kötter, T. Meinl, P. Ohl, K. Thiel, and B. Wiswedel, "KNIME - the Konstanz Information Miner: Version 2.0 and Beyond," *SIGKDD Explor. Newsl.*, vol. 11, no. 1, pp. 26–31, Nov. 2009. [Online]. Available: <http://doi.acm.org/10.1145/1656274.1656280>
- [23] C. M. Bishop, "Model-based machine learning," *Philosophical Transactions of the Royal Society A*, 2013.
- [24] T. Minka, J. Winn, J. Guiver, Y. Zaykov, D. Fabian, and J. Bronskill, "Infer.NET 0.3," 2018, microsoft Research Cambridge. <http://dotnet.github.io/infer>.
- [25] D. Breuker, "Towards model-driven engineering for big data analytics – an exploratory analysis of domain-specific languages for machine learning," 01 2014, pp. 758–767.
- [26] "ML-Quadrat," <https://github.com/arminmoin/ML-Quadrat>, accessed: 2020-09-12.
- [27] W. Newman, "A preliminary analysis of the products of HCI research, using pro forma abstracts," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ser. CHI '94. New York, NY, USA: Association for Computing Machinery, 1994, p. 278–284.
- [28] V. Basili, G. Caldiera, and H. D. Rombach, "The Goal Question Metric Approach," *Encyclopedia of Software Engineering - 2 Volume Set*, 1994.
- [29] J. Z. Kolter and M. J. Johnson, "REDD: A public data set for energy disaggregation research," in *Proceedings of the SustKDD workshop on Data Mining Applications in Sustainability*, 2011.
- [30] J. Kelly and W. Knottenbelt, "The UK-DALE dataset, domestic appliance-level electricity demand and whole-house demand from five UK homes," *Sci Data* 4, March 2015. [Online]. Available: <https://doi.org/10.1038/sdata.2015.7>
- [31] D. Murray, L. Stankovic, and V. Stankovic, "An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study," *Sci Data* 4, January 2017. [Online]. Available: <https://doi.org/10.1038/sdata.2016.122>
- [32] B. Selic, "A systematic approach to domain-specific language design using uml," 05 2007, pp. 2–9.
- [33] F. Lagarde, H. Espinoza, F. Terrier, and S. Gérard, "Improving uml profile design practices by leveraging conceptual domain models," in *Proceedings of the Twenty-Second IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 445–448. [Online]. Available: <https://doi.org/10.1145/1321631.1321705>